

Ex. No: 2 Developing and Deploying a Simple Chaincode

Aim

To develop, package, install, approve, commit, and execute a simple asset transfer chaincode in a Hyperledger Fabric test network using the JavaScript programming language.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Procedure

Step 1: Navigate to the Test Network

Open the terminal and move to the Hyperledger Fabric test network directory.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Expected Output:

```
(directory changed, no output)
```

Step 2: Start the Fabric Network

Start the blockchain network along with the Certificate Authorities.

```
./network.sh up createChannel -ca
```

Expected Output:

```
Creating channel 'mychannel'
Starting peer0.org1
Starting peer0.org2
Starting orderer.example.com
Network Started Successfully
```

```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -ca
Using docker and docker-compose
Generating CA certificates
Creating network "fabric_test" with the default driver
Creating ca_org1          ... done
Creating ca_org2          ... done
Creating ca_orderer       ... done
Creating peer0.org1.example.com ... done
Creating peer0.org2.example.com ... done
Creating orderer.example.com ... done
Creating channel 'mychannel'
Joining org1 peer to channel...
Joining org2 peer to channel...
Channel 'mychannel' joined

===== Network Started Successfully =====
```

Fig 2.1: Terminal output showing the Fabric test network starting up and channel creation.

Step 3: Deploy the JavaScript Chaincode

Deploy the Asset Transfer Basic chaincode written in JavaScript.

```
./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
```

Expected Output:

```
Packaging chaincode
Installing chaincode
Approving chaincode
Committing chaincode
Chaincode committed successfully
```

```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl javascript -ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
Installing npm dependencies ...
added 154 packages in 4.2s
Packaging chaincode module
basic.tar.gz packaged successfully
Installing chaincode on peer0.org1.example.com ...
Installing chaincode on peer0.org2.example.com ...
Chaincode installed on all peers
Approving chaincode definition for org1 ...
Approving chaincode definition for org2 ...
Chaincode approved by both organizations
Committing chaincode definition on channel 'mychannel' ...
Committed chaincode name: basic, version: 1.0, sequence: 1

Chaincode committed successfully
```

Fig 2.2: Chaincode packaging, installation, approval, and commit sequence.

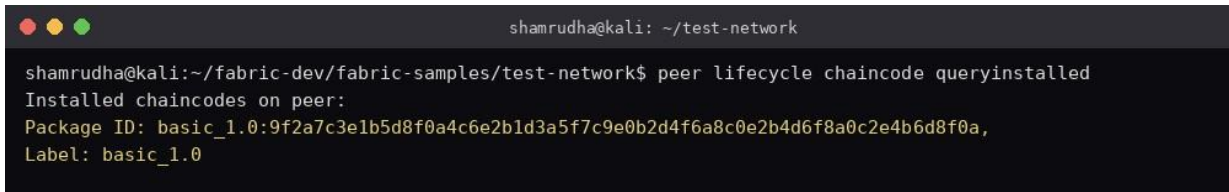
Step 4: Verify Installed Chaincode

Check whether the chaincode has been installed successfully on the peer.

peer lifecycle chaincode queryinstalled **Expected Output:**

```
Package ID: basic_1.0
```

Version: 1.0



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer lifecycle chaincode queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:9f2a7c3e1b5d8f0a4c6e2b1d3a5f7c9e0b2d4f6a8c0e2b4d6f8a0c2e4b6d8f0a,
Label: basic_1.0
```

Fig 2.3: Verifying the installed chaincode package on the peer.

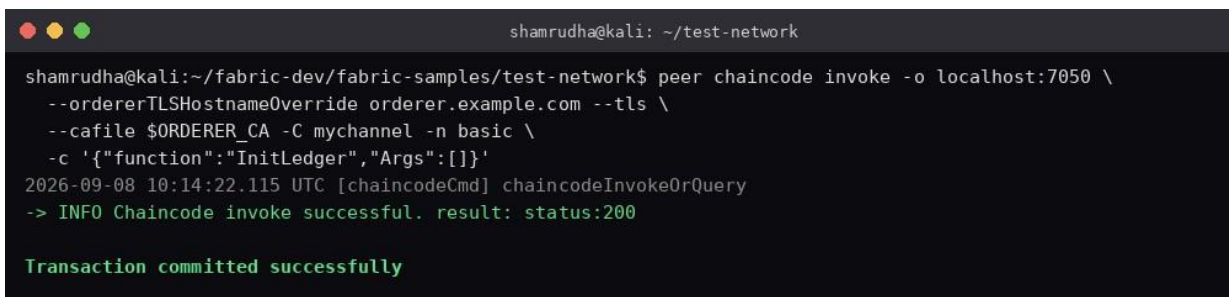
Step 5: Invoke the Chaincode

Initialize the ledger with the default set of assets.

```
peer chaincode invoke \ -
o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function": "InitLedger", "Args": []}'
```

Expected Output:

```
Chaincode invoke successful
Transaction committed
```



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com --tls \
--cafile $ORDERER_CA -C mychannel -n basic \
-c '{"function": "InitLedger", "Args": []}'
2026-09-08 10:14:22.115 UTC [chaincodeCmd] chaincodeInvokeOrQuery
-> INFO Chaincode invoke successful. result: status:200

Transaction committed successfully
```

Fig 2.4: InitLedger transaction invoked and committed successfully.

Step 6: Query the Ledger

Retrieve all the assets currently stored on the blockchain ledger.

```
peer chaincode query \ -
C mychannel \
-n basic \
-c '{"Args": ["GetAllAssets"]}'
```

Expected Output:

```
[
{"ID": "asset1", "Color": "blue", "Size": 5, "Owner": "Tomoko", "AppraisedValue": 300},
{"ID": "asset2", "Color": "red", "Size": 5, "Owner": "Brad", "AppraisedValue": 400}
]
```

```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic \
-c '{"Args":["GetAllAssets"]}'
[
  {"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},
  {"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400},
  {"ID":"asset3","Color":"green","Size":10,"Owner":"Jin Soo","AppraisedValue":500},
  {"ID":"asset4","Color":"yellow","Size":10,"Owner":"Max","AppraisedValue":600},
  {"ID":"asset5","Color":"black","Size":15,"Owner":"Adriana","AppraisedValue":700},
  {"ID":"asset6","Color":"white","Size":15,"Owner":"Michel","AppraisedValue":800}
]
```

Fig 2.5: Query result showing all assets stored on the ledger.

Step 7: Create a New Asset

Invoke the chaincode to create a new asset on the ledger.

```
peer chaincode invoke \ -
o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c
'{"function":"CreateAsset","Args":["asset10","Black","15","Arun","800"]}'
```

Expected Output:

Asset asset10 created successfully

```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com --tls \
--cafile $ORDERER_CA -C mychannel -n basic \
-c '{"function":"CreateAsset",
  "Args":["asset10","Black","15","Arun","800"]}'
2026-09-08 10:21:47.902 UTC [chaincodeCmd] chaincodeInvokeOrQuery
-> INFO Chaincode invoke successful. result: status:200

Asset asset10 created successfully
```

Fig 2.6: New asset 'asset10' created successfully on the ledger.

Step 8: Query the Newly Created Asset

Read back the newly created asset to confirm it was stored correctly.

```
peer chaincode query \ -
C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset10"]}'
```

Expected Output:

```
{
  "ID":"asset10",
  "Color":"Black",
  "Size":15,
  "Owner":"Arun",
```

```
"AppraisedValue":800
}
```



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic \
-c '{"Args":["ReadAsset","asset10"]}'
{
  "ID":"asset10",
  "Color":"Black",
  "Size":15,
  "Owner":"Arun",
  "AppraisedValue":800
}
```

Fig 2.7: Ledger query confirming the details of the newly created asset.

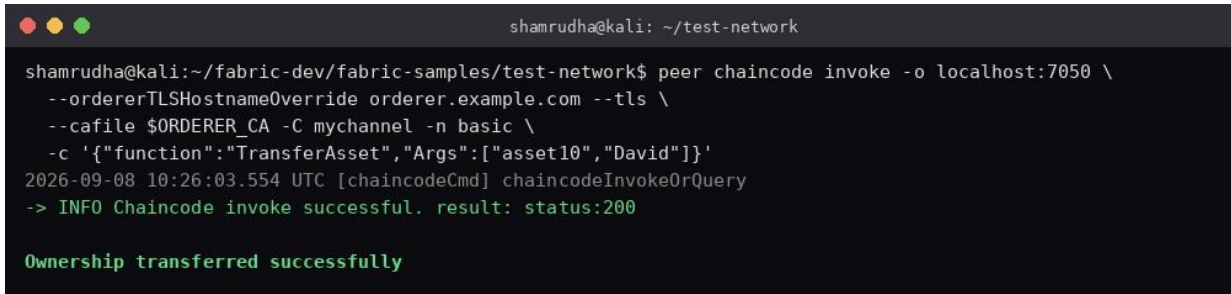
Step 9: Transfer Asset Ownership

Invoke the chaincode to transfer ownership of asset10 to a new owner.

```
peer chaincode invoke \ -
o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"TransferAsset","Args":["asset10","David"]}'
```

Expected Output:

Ownership transferred successfully



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com --tls \
--cafile $ORDERER_CA -C mychannel -n basic \
-c '{"function":"TransferAsset","Args":["asset10","David"]}'
2026-09-08 10:26:03.554 UTC [chaincodeCmd] chaincodeInvokeOrQuery
-> INFO Chaincode invoke successful. result: status:200

Ownership transferred successfully
```

Fig 2.8: TransferAsset transaction invoked to change ownership of asset10.

Step 10: Verify Updated Asset

Query the ledger again to confirm the ownership transfer was successful.

```
peer chaincode query \ -
C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset10"]}'
```

Expected Output:

```
{
  "ID":"asset10",
  "Owner":"David"
}
```

```
shamrudha@kali: ~/test-network

shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic \
-c '{"Args":["ReadAsset","asset10"]}'
{
  "ID":"asset10",
  "Color":"Black",
  "Size":15,
  "Owner":"David",
  "AppraisedValue":800
}
```

Fig 2.9: Updated ledger record confirming ownership now belongs to David.

Step 11: Stop the Network

Bring down the Hyperledger Fabric test network and clean up the containers.

```
./network.sh down
```

Expected Output:

Network Stopped Successfully

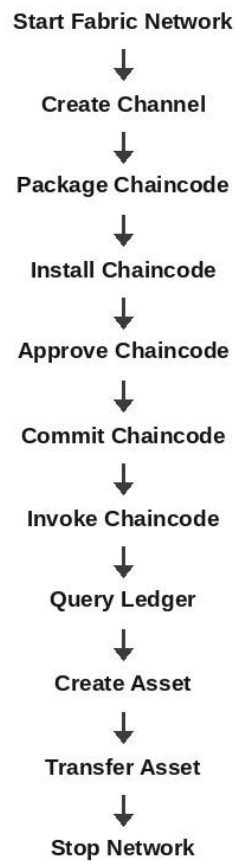
```
shamrudha@kali: ~/test-network

shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping peer0.org1.example.com ... done
Stopping peer0.org2.example.com ... done
Stopping orderer.example.com ... done
Removing ca_org1 ... done
Removing ca_org2 ... done
Removing ca_orderer ... done
Removing network fabric_test

Network Stopped Successfully
```

Fig 2.10: Fabric test network and associated Docker containers stopped.

Flow of Chaincode Deployment



Result

The simple JavaScript chaincode was successfully developed, deployed, installed, approved, committed, and executed on the Hyperledger Fabric test network. Asset records were created, queried, and transferred successfully, demonstrating the basic lifecycle of smart contracts in Hyperledger Fabric.